

Stack & Heap concepts in Java

→ JVM divides memory into stack & heap memory. whenever we declare new variables and objects, call a new methods, declare a string or perform similar operations, JVM designates memory to these operations from either stack or heap space.

Stack in Java is used for static memory allocation & execution of a thread. It contains primitive values that are specific to a method & references to objects referred from the method that are in a heap.

Key features

- ⊗ It grows & shrinks as new methods are called and returned, respectively.
- ⊗ Variable inside the stack exist only as long as the method that created them is running.
- ⊗ It automatically allocates & deallocates when the method finishes execution.
- ⊗ If this memory is full, Java throws `java.lang.StackOverflowError`
- ⊗ Access to this memory is fast compared to Heap memory.
- ⊗ This memory is thread safe, as each thread operates in its own stack.

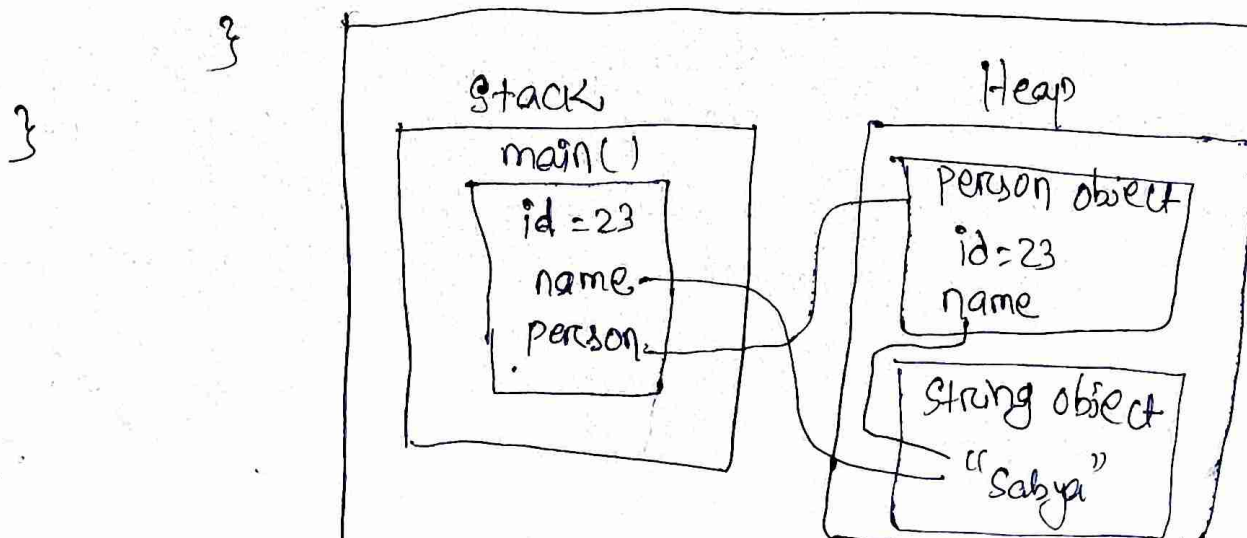
Heap → It is used for the dynamic memory allocation of Java objects and JRE classes at runtime.

- New objects are always created in heap space and reference to these objects are stored in stack memory.
- These objects have global access and we can access them from anywhere in the application.

ex code is in (Java/OOP/Stack-Heap.java)

```
class Person {  
    int id;  
    String name;  
    public Person (int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

```
public class Stack-Heap {  
    private static Person buildPerson (int id, String name) {  
        return new Person (id, name);  
    }  
    public static void main (String[] args) {  
        int id = 23;  
        String name = "Sabya";  
        Person person = null;  
        person = buildPerson (id, name);  
    }  
}
```



Key Features of Java (Heap):-

(*) It's accessed via complex memory management techniques that include the young Generation, old or Tenured Gen. and permanent Generation.

(*) If Heap space is full Java throws

`java.lang.OutOfMemoryError`

(*) Access memory slower compared to stack.

(*) It needs Garbage collector to free up unused objects so as to keep efficiency of the memory usage.

(*) A heap isn't thread safe and needs to be guarded by properly synchronizing the code.

⇒ we can break this memory model down into smaller parts, called Generation, which are.

(i) Young Generation → this is where all new objects are allocated and aged. A minor Garbage collection occurs when this fills up.

(ii) Old or Tenured Generation → this is where long surviving objects are stored. When objects are stored in the young Generation, a threshold for the objects age is set. And when that threshold is reached, the object is moved to the old Gen.

(iii) Permanent Generation → this consists of JVM metadata for the runtime classes and application methods.